SES - Group Project

Version: 20260519

This document outlines the requirements and milestones for the SES Group Project. The project is divided into two main parts, with presentations scheduled for **March 24th** and **TBD**, respectively.

Part 1: Security Assessment & Tooling Setup

Presentation Date: March 24

For this first part of the project, students must use the project baseline. **Crucially, you are not allowed to develop new features on top of the existing solution.** Your focus will be on analyzing and securing the given baseline.

Requirements

1. **Threat Modeling & Documentation**
    - Perform Threat Modeling, create Data Flow Diagrams (DFDs), construct Attack Trees, and write Abuse Stories.
    - **Repository Rules:**
        - Only the **Abuse Stories** should be created and stored within the main software repository.
        - The rest of the documentation (Threat Modeling, DFDs, Attack Trees) must be placed in a separate, **dedicated documentation repository** available at https://classroom.github.com/a/RWlODmUQ.
2. **Static Application Security Testing (SAST)**
    - Define and integrate SAST scanners across different levels of the development lifecycle into the software repository.
    - Examples of integration points include:
        - Pre-commit/Git hooks;
        - Pull Request (PR) checks / CI/CD pipelines;
        - Among others.
3. **Dynamic Application Security Testing (DAST)**

- Define and configure an appropriate environment to run DAST scanners against the application in the AppSec repository.

4. **Conduct a Real Assessment**
    - Use the setup scanners and methodologies to conduct a real security assessment of the baseline project.
    - Document the findings, vulnerabilities, and potential mitigation strategies based on the assessment results.

Part 2: Fixing Issues & Future Development

Presentation Date: TBD in the class

For this second part of the project, the constraint from Part 1 is lifted: students may evolve the baseline and introduce new security-relevant functionality. Building on the assessment carried out previously, the focus is on integrating security as an explicit architectural concern, hardening the API surface, formalizing identity and authorization, and introducing perimeter defenses, and on producing evidence that the resulting system is measurably more defensible than the one assessed in Part 1.

Requirements

1. **API Hardening Against OWASP API Top 10 (2023)**
    - Audit the application's REST endpoints and remediate them against the OWASP API Security Top 10 (2023).
    - Coverage must include, where the corresponding flow exists in the baseline:
        - Object-level authorization on every resource endpoint (API1 — BOLA);
        - Property-level authorization and response filtering, eliminating field leakage (API3 — BOPLA);
        - Resource consumption controls: rate limiting and request-size limits (API4);
        - Function-level authorization with deny-by-default on administrative routes (API5);
        - Security configuration hardening: HTTP security headers, secure cookie flags, removal of stack traces from production responses (API8);
        - API inventory via a generated OpenAPI specification, kept in the main software repository (API9).
    - Each implemented control must be traceable — via commit message or report entry — to the OWASP API item it addresses.

2. **Authentication & Identity Management**

- Replace any ad-hoc authentication present in the baseline with a token-based scheme:
  - Short-lived access tokens (JWT, signed with RS256);
  - Long-lived refresh tokens with rotation, reuse detection, and server-side revocation;
  - CSRF protection on any cookie-bearing flow.
- Integrate an external Identity Provider (e.g., Keycloak) using OpenID Connect (Authorization Code flow with PKCE).
- Map application roles to realm roles and validate tokens against the IdP's JWKS endpoint.
- Document the login, logout, refresh, and revocation flows.

3. **Explicit Authorization Layer**

- Implement Role-Based Access Control with enforcement at two layers:
  - Route-level — middleware or decorator restricting access by role;
  - Object-level — ownership and permission check in the data-access layer.
- Roles and permissions must be expressed as data (configuration file or database table), not embedded in controller logic.
- Provide unit tests covering each role × action × resource combination.

4. **Perimeter Defense**

- Place a reverse proxy and Web Application Firewall in front of the application (e.g., Nginx + ModSecurity, or Caddy + Coraza).
- Configure the OWASP Core Rule Set in blocking mode; document any tuning performed and the false positives it eliminated.
- TLS termination, request-size limits, and IP-based rate limiting must occur at the perimeter.

5. **Zero Trust Architecture Framing**

- Document the resulting system in terms of NIST SP 800-207, explicitly labeling:
  - The **Subject** (user and device);
  - The **Policy Enforcement Point(s)** — perimeter (WAF) and in-application (route- and object-level checks);
  - The **Policy Decision Point** — the authorization layer from Requirement 3;
  - The **Policy Administrator** — the Identity Provider from Requirement 2.
- Provide a mapping of the seven Zero Trust tenets (NIST SP 800-207 §2.1) to the project's implementation status, explicitly acknowledging tenets not yet met as future work.

6. **Re-Assessment & Evidence**

    - Re-run the SAST and DAST scanners established in Part 1 against the new build.
    - The DAST scan must be run **twice**: once against the application directly, and once through the perimeter WAF, so that the contribution of the WAF can be isolated.
    - Report before/after results, broken down by severity and rule category.
    - Perform an **abuse-story regression**: for each abuse story written in Part 1, classify the current status as *prevented*, *detected*, or *residual risk*, and justify the classification.

Deliverables

- **Main software repository:** updated source code with security commits clearly labeled, updated Abuse Stories, and the generated OpenAPI specification.
- **Dedicated documentation repository:** updated DFDs and threat model reflecting the new architecture; the Zero Trust architecture diagram and NIST SP 800-207 tenet mapping; SAST and DAST exports (both DAST runs); the Part 2 report; and the presentation deck.

Repository Rules

The repository rules from Part 1 remain in effect: code and Abuse Stories live in the main software repository; threat modeling artifacts, architecture documentation, scanner evidence, and the final report belong in the dedicated documentation repository.